



INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO CEARÁ
IFCE CAMPUS UMIRIM

CURSO: S2 - ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

DISCIPLINA: ENGENHARIA DE SOFTWARE

PROFESSORA: RONEY REIS

ALUNOS: VICTOR MANUEL CUNHA SANTOS E JOÃO EMANUEL CARLOS LIMA

CONTROLE DE ESTOQUE

1. VISÃO E ESCOPO

- **Problema:** Atualmente, a falta de um sistema centralizado faz com que a empresa perca dinheiro com produtos parados ou, pior, perca vendas porque o material acabou e ninguém viu;
- **Objetivos:**
 - Registrar todas as entradas e saídas de mercadoria sem erro;
 - Impedir que o estoque fique negativo através de travas no código;
 - Avisar o gerente na hora quando um produto chegar no limite mínimo.
- **Escopo IN:** Cadastro de itens, controle de movimentação (quem fez e quando), login com JWT e relatório de itens críticos;
- **Escopo OUT:** O sistema não vai emitir nota fiscal nem fazer pagamentos automáticos para fornecedores neste momento.

2. STAKEHOLDERS E PERSONAS

- **Stakeholders:** Dono da Loja, Estoquista, Vendedor, Gerente de TI e Fornecedores;
- **Persona 1 - "Seu Cauã" (Estoquista):** Ele trabalha no depósito e quer algo simples. A dor dele é esquecer de anotar o que sai. O objetivo é dar baixa no material pelo tablet rapidinho.
- **Persona 2 - "Roney" (Gerente):** Ele precisa saber o que comprar. A dor dele é o estoque furado. O objetivo é tirar o relatório de "baixo estoque" e já fazer o pedido.

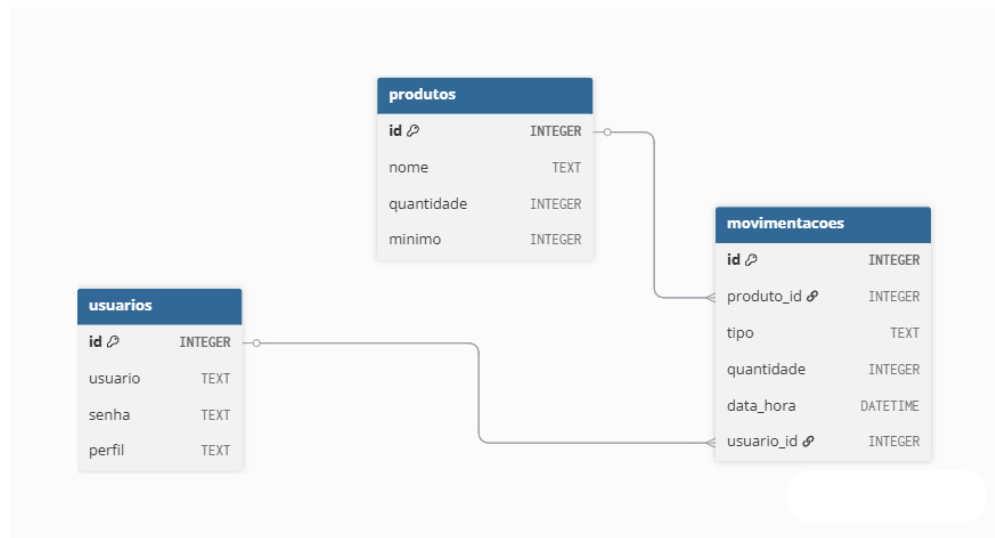
3. REQUISITOS FUNCIONAIS (RF) E NÃO FUNCIONAIS (RNF)

- **RF01:** O sistema deve exigir login com token JWT para qualquer operação;
- **RF02:** O sistema deve registrar quem fez a movimentação (ID do usuário) e o horário;
- **RF03:** O sistema deve bloquear saídas se a quantidade pedida for maior que o saldo.
- **RNF01 (Portabilidade):** O sistema tem que rodar em containers Docker para funcionar em qualquer PC;
- **RNF02 (Segurança):** O acesso é separado por perfis (admin, estoquista e consulta);
- **RNF03 (Confiabilidade):** O banco de dados SQLite deve ser persistente através de volumes no Docker.

4. HISTÓRIAS DE USUÁRIO

- "Como Estoquista, quero registrar que chegaram 50 sacos de cimento para que o estoque atualize sozinho."
- "Como Vendedor, quero consultar se tem o produto para não prometer o que não tem pro cliente."
- "Como Gerente, quero ver o relatório de itens acabando para fazer a compra antes do estoque zerar."

5. MODELO DE DADOS (ER)



- **Usuários:** id, usuário, senha, perfil;
- **Produtos:** id, nome, quantidade total, estoque mínimo;
- **Movimentações:** id, produto_id, tipo (entrada/saída), quantidade, data, usuario_id.

6. PROTÓTIPO

- **Tela de Login:** Autenticação do utilizador;
- **Lista de Produtos:** Visão geral com quantidades atuais;
- **Formulário de Movimentação:** Escolha do tipo (I/O) e quantidade;
- **Relatório de Itens Críticos:** Listagem destacada de produtos com stock baixo;
- **Gestão de Utilizadores:** Ecrã exclusivo para o administrador criar novos acessos.

7. ARQUITETURA

O sistema utiliza uma arquitetura baseada em Micro-serviços com persistência de dados.

O fluxo segue:

Cliente (Postman/Navegador) -> Servidor Node.js (API Express) -> Base de Dados SQLite (Volume Docker).